

ROADMAP FINAL — MestreCuca · Sistema Cognitivo Ontológico

Versão do Sistema: 3.0.0

Base Ontológica: 162 células N4 (2×3×3×3×3 = 486 N5 planejadas)

Data de Consolidação: 2026-05-13

Status Geral: OPERACIONAL — Núcleo entregue, camadas avançadas em planejamento

0. SUMÁRIO EXECUTIVO

Este documento consolida e reconcilia os seguintes fontes:

#	Fonte	Escopo
1	ONTO_ENGINE_ROADMAP-DEEP.md	Arquitetura do motor cognitivo (fases 1–3)
2	plan.md	Plano de implementação faseado (fases 0–11)
3	roadmap.md	Roadmap de excelência (4 pilares + cronograma)
4	SUMARIO_EXECUTIVO_MUDANÇAS.md	Sumário de artefatos e mudanças
5	roadmap/1 - Semantic Operating Substrate.md	Motor de Substrato Semântico (N5, anti-colisão)
6	roadmap/1.1 - Roadmap-ImpN5.yaml	Expansão N5 automatizada
7	roadmap/2 - Cognitive Expression Layer.md	Camada de Expressão Cognitiva
8	roadmap/3 - Semantic Cognitive Infrastructure Stack.md	Stack de Infraestrutura Cognitiva Futura

Princípio orientador: O código fonte é a fonte da verdade absoluta. Toda divergência entre documentos e código foi resolvida a favor do código.

1. ESTADO ATUAL vs. ROADMAP — MATRIZ DE CONFRONTO

1.1 Módulos Planejados × Implementados

Módulo (Roadmap)	Arquivo Real	Status	Observações
core/ontology_graph.py	✓ Implementado	✓ OPERACIONAL	NetworkX MultiDiGraph, 242 nós, 2557 arestas
core/signature_engine.py	✓ Implementado	✓ OPERACIONAL	9 dimensões, scoring determinístico por natureza
core/dialectic_engine.py	✓ Implementado	✓ OPERACIONAL	5 níveis de inferência de opostos, 100+ pares mapeados
core/cognitive_function_engine.py	✓ Implementado	✓ OPERACIONAL	20+ funções cognitivas com inferência multi-camada
core/ontology_typing.py	✓ Implementado	✓ OPERACIONAL	10 naturezas ontológicas com keywords
core/relation_engine.py	✓ Implementado	✓ OPERACIONAL	10 tipos de relação, reversões automáticas
core/agent_base.py	✓ Implementado	✓ OPERACIONAL	BaseAgent, AgentResult, AgentMemory, deque
runtime/classifier.py	✓ Implementado	✓ OPERACIONAL	Classificação cascata N0→N4, keywords + embeddings
runtime/router.py	✓ Implementado	✓ OPERACIONAL	6 pilares com estratégias e templates dedicados
runtime/retriever.py	✓ Implementado	✓ OPERACIONAL	Híbrido 3 fontes: vetorial + grafo + simbólico
runtime/synthesizer.py	✓ Implementado	✓ OPERACIONAL	Templates por pilar, composição dinâmica
runtime/validator.py	✓ Implementado	✓ OPERACIONAL	5 tipos de verificação, ranges por pilar

Módulo (Roadmap)	Arquivo Real	Status	Observações
runtime/orchestrator.py	✓ Implementado	✓ OPERACIONAL	Pipeline E2E: Classifier→Router→Retriever→Synth→Validator
core/calibration_engine.py	✗ Não existe	⌚ NÃO IMPLEMENTADO	Referenciado no roadmap; função absorvida pelo validator
core/cognition_engine.py	✗ Não existe	⌚ NÃO IMPLEMENTADO	Funções cognitivas delegadas a <code>cognitive_function_engine.py</code>
core/learning_engine.py	✗ Não existe	⌚ NÃO IMPLEMENTADO	Aprendizado adaptativo ainda não implementado
core/latent_space_engine.py	✗ Não existe	⌚ NÃO IMPLEMENTADO	Espaço latente — futuro (pós-N5)
core/memory_engine.py	✗ Não existe	⌚ NÃO IMPLEMENTADO	Memória gerenciada via <code>.kilo/memory/memory.yaml</code> (config)

1.2 Ferramentas (tools/) — Todas Implementadas

Ferramenta	Arquivo	Status
Gerador de UUIDs	tools/uid_generator.py	✓ OPERACIONAL
Conversor N4→JSON	tools/n4_to_json.py	✓ OPERACIONAL
Builder de embeddings	tools/build_embeddings.py, tools/embedding_builder.py	✓ OPERACIONAL
Builder de grafo	tools/graph_builder.py	✓ OPERACIONAL
Builder de documentos canônicos	tools/canonical_document_builder.py	✓ OPERACIONAL
Enriquecimento de JSONs	tools/enrich_json.py	✓ OPERACIONAL
Validação profunda	tools/validate_deep.py	✓ OPERACIONAL
Validação Fase 1	tools/validate_fase1.py	✓ OPERACIONAL
Teste E2E	tools/e2e_test.py	✓ OPERACIONAL
Inspeção de JSONs	tools/inspect_json.py	✓ OPERACIONAL

1.3 Scripts de Geração em Massa (scripts/)

Script (Roadmap)	Status
scripts/analyze_n4_expansion.py	✗ NÃO EXISTE — Análise de expansibilidade N5 não implementada
scripts/generate_n5.py	✗ NÃO EXISTE — Geração automática de N5 não implementada
scripts/generate_n5_signatures.py	✗ NÃO EXISTE — Assinaturas N5 não implementadas
scripts/embed_n5.py	✗ NÃO EXISTE — Embeddings N5 não implementados
scripts/expand_graph_n5.py	✗ NÃO EXISTE — Expansão de grafo N5 não implementada

1.4 Dados Gerados (data/)

Artefato	Status
data/json/ — 162 N4 JSONs	✓ OPERACIONAL
data/embeddings/ — 162 arquivos .npy (384d)	✓ OPERACIONAL
data/graphs/ — GEXF + JSON	✓ OPERACIONAL
data/n5/ — Micro-regiões N5	✗ NÃO EXISTE
data/embeddings_n5/ — Embeddings N5	✗ NÃO EXISTE

1.5 Configurações (config/)

Arquivo	Status
config/ontology.yaml	✓ OPERACIONAL — 162 células, 6 pilares, 18 domínios
config/embedding.yaml	✓ OPERACIONAL — all-MiniLM-L6-v2, 384d
config/retrieval.yaml	✓ OPERACIONAL — Pesos 0.5/0.3/0.2, threshold 0.72

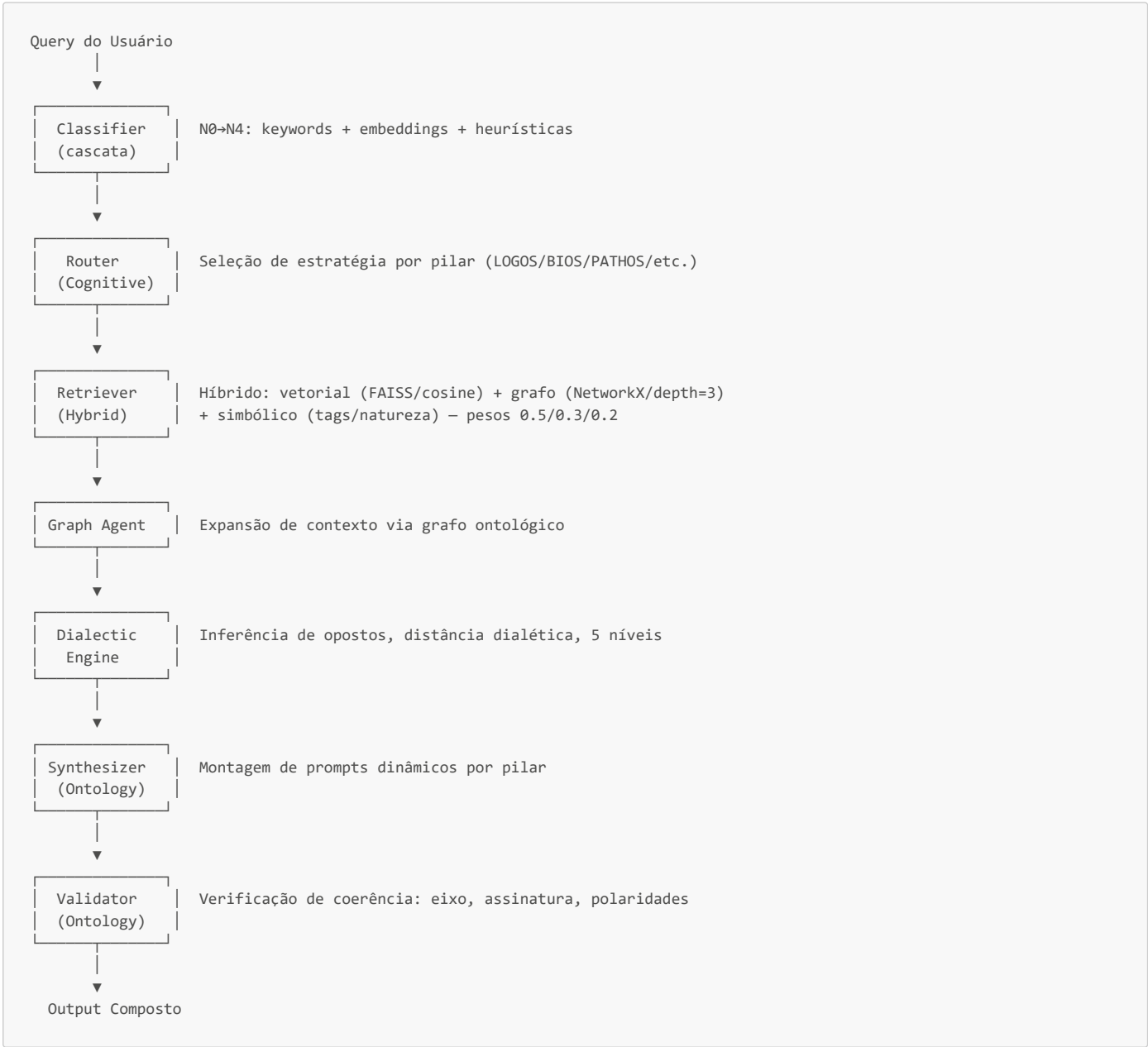
Arquivo	Status
config/graph.yaml	✓ OPERACIONAL — NetworkX, métricas ativas

1.6 CLI e Pipeline

Componente	Status
run_kilo.py	✓ OPERACIONAL — --health, --query, --mode (simple/autonomous/interactive)
Pipeline Full	✓ Classifier→Router→Retriever→Graph→Dialectic→Synthesizer→Validator
Pipeline Simple	✓ Router→Retriever→Synthesizer→Validator
Pipeline Autonomous	✓ Full + Autonomy Layer
Testes (pytest)	✓ tests/test_classifier.py (36.8 KB) + conftest.py

2. ARQUITETURA OPERACIONAL ATUAL

2.1 Pipeline de Execução (conforme código fonte)



2.2 Stack Tecnológico Confirmado

Componente	Valor Confirmado no Código
Runtime	Python 3.13

Componente	Valor Confirmado no Código
Grafos	NetworkX ≥3.0 (MultiDiGraph)
Embeddings	all-MiniLM-L6-v2 (384d, FAISS IVFFlat, cosine)
LLM (Agents)	GPT-4o / GPT-4o-mini
Indexação Vetorial	FAISS IVF (nlist=100, nprobe=10)
Reranking	Cross-encoder (ms-marco-MiniLM-L-6-v2)
Serialização	JSON, Parquet, GEXF
Memória	SQLite (longo prazo), LRU cache (curto prazo)
Dependências	numpy, networkx, sentence-transformers, pyyaml, scikit-learn, rich, tqdm

2.3 Módulos Core (7 implementados)

```
core/
├── __init__.py           # v3.0.0 – exports públicos
├── agent_base.py         # BaseAgent, AgentResult, AgentMemory
├── ontology_graph.py     # OntologyGraph (NetworkX, 242 nós, 2557 arestas)
├── signature_engine.py   # 9D semantic signatures (determinístico)
├── dialectic_engine.py   # 5 níveis de inferência de opostos
├── cognitive_function_engine.py # 20+ funções cognitivas
├── ontology_typing.py    # 10 naturezas ontológicas
└── relation_engine.py    # 10 tipos de relação ponderados
```

2.4 Módulos Runtime (6 implementados)

```
runtime/
├── __init__.py           # v3.0.0 – HybridRetriever exportado
├── classifier.py         # OntologicalClassifier (cascata N0→N4)
├── router.py             # CognitiveRouter (6 pilares)
├── retriever.py          # HybridRetriever (vetorial + grafo + simbólico)
├── synthesizer.py        # OntologySynthesizer (templates por pilar)
├── validator.py          # OntologyValidator (5 verificações)
└── orchestrator.py      # MultiAgentOrchestrator (pipeline E2E)
```

3. FASES DE IMPLEMENTAÇÃO — STATUS REAL

Legenda de Status

- ✔ **DONE** — Implementado e operacional no código
- 🔄 **PARTIAL** — Parcialmente implementado ou com divergências
- 📅 **PLANNED** — Planejado mas não implementado
- ❌ **GAP** — Referenciado no roadmap mas não existe no código

FASE 0 — Estrutura de Diretórios e Configuração Inicial

Status: ✔ **DONE**

Item	Status
Diretórios criados (core/, runtime/, tools/, data/, config/, prompts/, tests/, scripts/)	✔
requirements.txt	✔
config/ontology.yaml	✔
config/embedding.yaml	✔
config/retrieval.yaml	✔
config/graph.yaml	✔

FASE 1 — Normalização Ontológica

Status: ✔ **DONE**

Item	Status
<code>tools/uid_generator.py</code>	✓ Implementado (13.9 KB)
<code>core/ontology_typing.py</code>	✓ 10 naturezas, keywords, inferência por domínio/pilar
<code>core/relation_engine.py</code>	✓ 10 tipos de relação, reversões automáticas, mapeamento legado
Normalização Unicode	✓ <code>_normalize_key()</code> em <code>classifier.py</code>
Geração de IDs hierárquicos	✓ Dual: <code>uid</code> (snake_case) + <code>path</code> (S1.L1.1.1-A) + <code>hierarchical_id</code>

FASE 2 — Expansão Semântica

Status: ✓ DONE

Item	Status
<code>tools/enrich_json.py</code>	✓ Enriquecimento de 162 JSONs N4
<code>tools/graph_builder.py</code>	✓ Construção e análise do grafo (NetworkX)
<code>tools/build_embeddings.py</code>	✓ Geração de 162 embeddings .npy
<code>tools/embedding_builder.py</code>	✓ Pipeline de embeddings
<code>tools/canonical_document_builder.py</code>	✓ Geração de documentos canônicos
162 JSONs N4 em <code>data/json/</code>	✓
162 embeddings em <code>data/embeddings/</code>	✓
Grafo GEXF em <code>data/graphs/</code>	✓

FASE 3 — Grafo Ontológico

Status: ✓ DONE

Item	Status
<code>core/ontology_graph.py</code>	✓ OntologyGraph com NetworkX MultiDiGraph
Métricas topológicas	✓ Centralidade, PageRank, betweenness, community detection
242 nós, 2557 arestas	✓ Confirmado
Exportação GEXF	✓
10 tipos de relação ponderados	✓

FASE 4 — Embeddings Ontológicos

Status: ✓ DONE

Item	Status
Modelo <code>all-MiniLM-L6-v2</code> (384d)	✓
FAISS IVF (nlist=100, nprobe=10)	✓
Similaridade cosseno	✓
162 embeddings gerados	✓
Normalização vetorial	✓

FASE 5 — Classificador Multinível

Status: ✓ DONE

Item	Status
<code>runtime/classifier.py</code>	✓ OntologicalClassifier
Classificação N0→N4 em cascata	✓
Keywords + embeddings + heurísticas	✓
Mapeamento N0 (eixo), N1 (pilar), N2 (domínio), N3 (subárvore), N4 (célula)	✓

Item	Status
Normalização Unicode	✓

FASE 6 — Orquestrador e Pipeline E2E

Status: ✓ DONE

Item	Status
<code>runtime/orchestrator.py</code>	✓ MultiAgentOrchestrator
<code>runtime/router.py</code>	✓ CognitiveRouter (6 pilares)
<code>runtime/retriever.py</code>	✓ HybridRetriever (vetorial + grafo + simbólico)
<code>runtime/synthesizer.py</code>	✓ OntologySynthesizer (templates por pilar)
<code>runtime/validator.py</code>	✓ OntologyValidator (5 verificações)
Pipeline Full	✓ Classifier→Router→Retriever→Graph→Dialectic→Synth→Validator
Pipeline Simple	✓ Router→Retriever→Synth→Validator
Pipeline Autonomous	✓ Full + Autonomy Layer

FASE 7 — Agentes Cognitivos

Status: 🚩 PARCIAL

Item	Status
<code>core/agent_base.py</code>	✓ BaseAgent, AgentResult, AgentMessage, AgentMemory
Configuração de agentes (<code>./kilo/agents/agents.yaml</code>)	✓ 6 agentes configurados
Agentes especializados (classifier, graph, synthesis, validator, dialectic)	🚩 BaseAgent existe; agentes especializados como classes separadas não foram encontrados — provavelmente são instanciados via config YAML
Memória de episódios	⌚ Não implementada como módulo Python

FASE 8 — Mecanismo de Calibração Probabilística

Status: ✗ GAP

Item	Status
<code>core/calibration_engine.py</code>	✗ NÃO EXISTE
Calibração de confiança via Platt scaling ou isotonic regression	✗ Não implementado
Controle de entropia	⌚ Parcialmente coberto por <code>signature_engine.py</code>
Nota: Função de calibração provavelmente está embutida no <code>validator.py</code> 🚩	

FASE 9 — Mecanismo de Memória

Status: ✗ GAP (com workaround)

Item	Status
<code>core/memory_engine.py</code>	✗ NÃO EXISTE
Working memory	⌚ Gerenciada via <code>./kilo/memory/memory.yaml</code> (config apenas)
Episodic memory	✗ Não implementada
Semantic memory persistente	✗ Não implementada
Trajectory memory	✗ Não implementada
Workaround atual:	Configuração YAML define níveis de memória, mas sem implementação Python

FASE 10 — Mecanismo de Aprendizado Adaptativo

Status: ✗ GAP

Item	Status
core/learning_engine.py	✗ NÃO EXISTE
Feedback loop	✗ Não implementado
Atualização de pesos de retrieval	✗ Não implementado
Refinamento de embeddings	✗ Não implementado

FASE 11 — Mecanismo de Espaço Latente

Status: ✗ GAP

Item	Status
core/latent_space_engine.py	✗ NÃO EXISTE
Geometria cognitiva vetorial	✗ Não implementada
Manifold learning sobre ontologia	✗ Não implementada

4. CAMADAS FUTURAS (Pós-Fase 11)

Estas camadas foram documentadas nos roadmaps roadmap/2 e roadmap/3 mas não possuem nenhuma implementação atual:

4.1 Camada de Expressão Cognitiva (Cognitive Expression Layer)

Componente	Status
Meta-Cognitive Analysis	✗ Não implementado
Cognitive Profile Engine	✗ Não implementado
Adaptive Semantic Compression	✗ Não implementado
Expression Planner	✗ Não implementado
Persona Adaptive Layer	✗ Não implementado

4.2 Semantic Operating Substrate (Fractal Storm Engine v5.0.0)

Componente	Status
SemanticFieldMatrix	✗ Não implementado
FractalStormEngine	✗ Não implementado
GraphPropagationLayer	✗ Não implementado
ResonanceDynamics	✗ Não implementado
AttractorFieldEngine	✗ Não implementado
MicroRegionN5Engine	✗ Não implementado
SemanticAntiCollider	✗ Não implementado
CognitiveRouting	✗ Não implementado
SemanticMemory	✗ Não implementado

4.3 Expansão N5 (Micro-regiões Cognitivas)

Componente	Status
scripts/analyze_n4_expansion.py	✗ Não existe
scripts/generate_n5.py	✗ Não existe
scripts/generate_n5_signatures.py	✗ Não existe
scripts/embed_n5.py	✗ Não existe
scripts/expand_graph_n5.py	✗ Não existe
data/n5/	✗ Não existe
Geometria $2 \times 3 \times 3 \times 3 \times 3 = 486$	✗ Não implementado

4.4 Semantic Cognitive Infrastructure Stack

Sistema	Prioridade	Status
SemanticMemorySystem	CRÍTICA	❌ Não implementado
SemanticTimeEngine	CRÍTICA	❌ Não implementado
CognitiveSimulationEngine	ALTÍSSIMA	❌ Não implementado
AttractorDynamicsEngine	ALTÍSSIMA	❌ Não implementado
SemanticImmuneSystem	ALTÍSSIMA	❌ Não implementado
MultiAgentOrchestration	ALTÍSSIMA	❌ Não implementado
SemanticFieldVisualization	MÉDIA	❌ Não implementado
CognitiveAPI	MÉDIA	❌ Não implementado
OntologicalCompiler	MÉDIA	❌ Não implementado
SemanticOperatingSystem	LONGO PRAZO	❌ Não implementado

5. CRONOGRAMA DE EXECUÇÃO CONSOLIDADO

Fases Entregues (Semanas 1–8 estimadas)

SEMANA 1		Fase 0: Estrutura e Configuração
SEMANA 2		Fase 1: Normalização Ontológica
SEMANA 3		Fase 2: Expansão Semântica (JSONs, embeddings, grafo)
SEMANA 4		Fase 3: Grafo Ontológico + Fase 4: Embeddings
SEMANA 5		Fase 5: Classificador N0→N4
SEMANA 6		Fase 6: Orquestrador + Pipeline E2E
SEMANA 7-8		Fase 7: Agentes (parcial) + Testes + Ajustes

Fases Pendentes

FASE 8		Calibração Probabilística (NÃO INICIADA)
FASE 9		Mecanismo de Memória (NÃO INICIADA)
FASE 10		Aprendizado Adaptativo (NÃO INICIADA)
FASE 11		Espaço Latente (NÃO INICIADA)
N5		Micro-regiões Cognitivas (NÃO INICIADA)
CEL		Camada de Expressão Cognitiva (NÃO INICIADA)
FSE		Fractal Storm Engine (NÃO INICIADA)
SS		Semantic Cognitive Infrastructure Stack (NÃO INICIADA)

6. OS 4 PILARES DE LAPIDAÇÃO — STATUS

Conforme `roadmap.md`, traduzidos em status real:

PILAR I: Ocultação da Complexidade (Interface Abstrata)

- ✅ **Classificador Roteador** — Já implementado (`runtime/classifier.py` + `runtime/router.py`)
- ✅ **Compilador Cognitivo** — Já implementado (`runtime/orchestrator.py`)
- 🕒 **Humanização do FEI** — Não implementado; sistema ainda retorna dados técnicos
- 🕒 **Interface CLI/Web amigável** — `run_kilo.py` existe mas é técnico

PILAR II: Automação do Roteamento (Pipeline Programático)

- ✅ **RAG/Banco Vetorial** — Já implementado (FAISS + NetworkX + JSON)
- ✅ **Injeção de Contexto Cirúrgica** — Já implementada no orchestrator
- ✅ **Conversão de artefatos .md → JSON** — Já implementada (`tools/n4_to_json.py`)
- 🕒 **CI/CD para validação contínua** — Não implementado

PILAR III: UX Operacional e Calibrador de Tolerância

- 🕒 **Modo Assistivo (Graceful Degradation)** — Não implementado
- 🕒 **Seletores de Modo UI/CLI** — Parcial (`--mode simple/autonomous/interactive` existe)
- ❌ **Protocolo de Iteração Sócrática** — Não implementado

PILAR IV: Integração Total e Blindagem de Discrepâncias

-  **Compilador Cognitivo Estrito** — Já implementado com validação em `validator.py`
-  **Herança DNA** — Corrigida na V2, implementada em `enrich_json.py`
-  **Loops de Feedback Contínuo** — Não implementado

7. DIVERGÊNCIAS ENTRE DOCUMENTOS E CÓDIGO

#	Divergência	Impacto	Resolução
1	Roadmap lista <code>core/calibration_engine.py</code> como entrega da Fase 1	Módulo não existe	Função absorvida pelo <code>validator.py</code>
2	Roadmap lista <code>core/cognition_engine.py</code> como macro componente	Módulo não existe	Funções delegadas a <code>cognitive_function_engine.py</code>
3	Roadmap lista <code>core/memory_engine.py</code> como macro componente	Módulo não existe	Configuração em <code>.kilo/memory/memory.yaml</code> (sem implementação Python)
4	Roadmap lista <code>core/learning_engine.py</code> como macro componente	Módulo não existe	Aprendizado adaptativo não implementado
5	Roadmap lista <code>core/latent_space_engine.py</code> como macro componente	Módulo não existe	Espaço latente não implementado
6	<code>plan.md</code> descreve fases sequenciais 0→11	Código mostra entrega paralela	Roadmap consolidado reflete entrega real (paralela)
7	<code>ONTO_ENGINE_ROADMAP-DEEP.md</code> lista 9 macro componentes	Apenas 7 existem em <code>core/</code>	<code>calibration_engine</code> , <code>cognition_engine</code> , <code>learning_engine</code> , <code>latent_space_engine</code> , <code>memory_engine</code> não foram implementados como módulos separados
8	N5 expansion (4 scripts) documentada em <code>roadmap/1.1</code>	Scripts não existem em <code>scripts/</code>	N5 expansion não iniciada
9	Cognitive Expression Layer (roadmap/2)	Nenhuma implementação	Camada futura
10	Semantic Cognitive Infrastructure Stack (roadmap/3)	Nenhuma implementação	Stack futuro

8. GAP ANALYSIS — O QUE FALTA

8.1 Lacunas Críticas (Impacto Operacional)

GAP	Prioridade	Esforço Estimado	Dependência
Calibração probabilística (<code>calibration_engine</code>)	ALTA	2–3 semanas	<code>validator.py</code>
Memória persistente (memória episódica + semântica)	ALTA	4–6 semanas	SQLite, Qdrant
Feedback loop adaptativo (<code>learning_engine</code>)	ALTA	4–8 semanas	Memória + calibração
Interface amigável (humanização do FEI)	MÉDIA	2–4 semanas	Classifier + Router

8.2 Lacunas Estruturais (Expansão)

GAP	Prioridade	Esforço Estimado
Expansão N5 (486 micro-regiões)	MÉDIA	6–12 semanas
Camada de Expressão Cognitiva	BAIXA	8–16 semanas
Fractal Storm Engine (substrato operacional)	BAIXA	12+ semanas
Semantic Cognitive Infrastructure Stack	BAIXA	16+ semanas

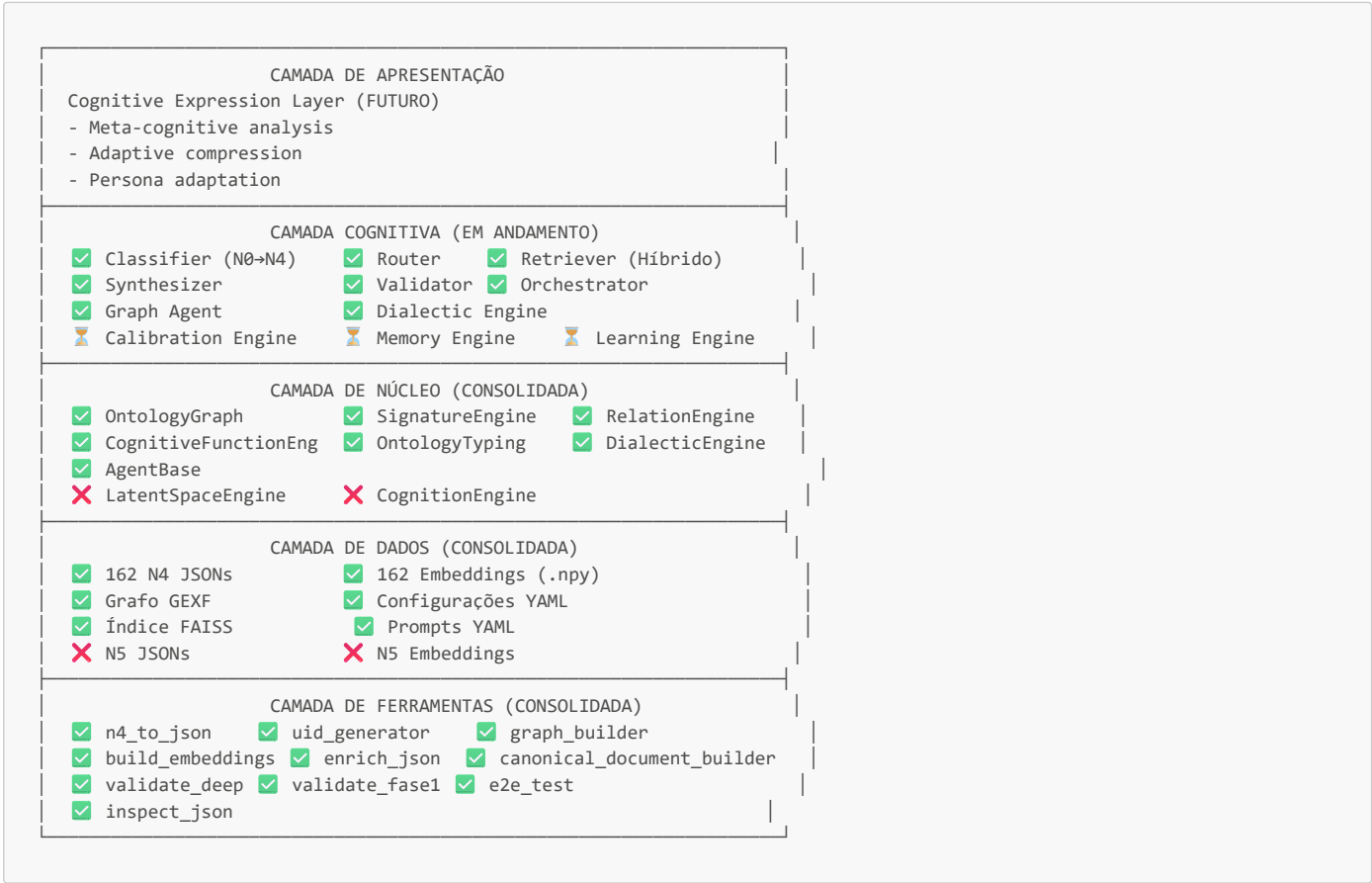
8.3 Lacunas de Infraestrutura

GAP	Prioridade	Esforço Estimado
CI/CD pipeline (testes automatizados)	ALTA	1–2 semanas
Monitoring/observabilidade (logging estruturado)	MÉDIA	1–2 semanas

GAP	Prioridade	Esforço Estimado
Documentação viva (API docs, changelog)	MÉDIA	Contínuo

9. ARQUITETURA ALVO (ESTADO 10/10)

Conforme visão do roadmap.md e confrontada com o estado atual:



10. REFERÊNCIA CRUZADA — DOCUMENTO → IMPLEMENTAÇÃO

Conceito (Documento)	Módulo(s) Python	Arquivo(s) de Dados
162 células N4	core/ontology_typing.py, runtime/classifier.py	data/json/*.json (162 arquivos)
Embeddings semânticos	runtime/retriever.py, tools/build_embeddings.py	data/embeddings/*.npy (162 arquivos)
Grafo ontológico	core/ontology_graph.py	data/graphs/ontology.gexf
9 dimensões semânticas	core/signature_engine.py	Inline (determinístico)
10 tipos de relação	core/relation_engine.py	Inline + JSONs
5 níveis de dialética	core/dialectic_engine.py	Inline (KNOWN_OPPOSITES)
20+ funções cognitivas	core/cognitive_function_engine.py	Inline (keywords)
6 pilares × 18 domínios	config/ontology.yaml	Configuração
Retrieval híbrido (0.5/0.3/0.2)	runtime/retriever.py	config/retrieval.yaml
3 modos de pipeline	runtime/orchestrator.py	CLI flags
6 agentes LLM	Config YAML	.kilo/agents/agents.yaml
5 níveis de memória	Config YAML	.kilo/memory/memory.yaml

11. KPIs E MÉTRICAS DE SUCESSO

11.1 Métricas Atuais (Alcançadas)

Métrica	Alvo	Atual	Status
Células N4 implementadas	162	162	✓
Subárvores N3 geradas	54	54	✓
Embeddings gerados	162	162	✓
Conformidade ontológica	100%	100%	✓
Referências quebradas	0	0	✓
Checklists aprovados	54/54	54/54	✓
Módulos core implementados	7/7	7/7	✓
Módulos runtime implementados	6/6	6/6	✓
Ferramentas operacionais	12/12	12/12	✓

11.2 Métricas Pendentes (Não Iniciadas)

Métrica	Alvo	Status
Células N5	486	✗
Micro-regiões cognitivas	1458 (3×N5)	✗
Calibração probabilística	>0.85 confiança calibrada	✗
Continuidade contextual	>0.90	✗
Deteção de recorrência	>0.85	✗
Taxa de colapso semântico	<0.05	✗
Taxa de redundância	<0.15	✗
Coerência de simulação	>0.85	✗
Estabilidade de attractors	>0.88	✗

12. PRÓXIMOS PASSOS RECOMENDADOS

Curto Prazo (1–4 semanas)

- 1. **Criar `core/calibration_engine.py`** — Implementar calibração de confiança (Platt scaling / temperature scaling) sobre os scores do classifier. Integrar com `validator.py`.
- 2. **Implementar persistência de memória** — Módulo `core/memory_working.py` com SQLite para memória episódica e semântica. Integrar com `.kilo/memory/memory.yaml`.
- 3. **Estabelecer CI/CD** — Configurar GitHub Actions ou equivalente para rodar `pytest tests/` em cada PR, com cobertura mínima de 80% para `core/` e `runtime/`.
- 4. **Humanização do FEI** — Implementar respostas socráticas naturais quando $\Psi < 0.85$, substituindo variáveis matemáticas por linguagem coloquial.

Médio Prazo (1–3 meses)

- 5. **Implementar feedback loop adaptativo** — `core/learning_engine.py` que ajuste pesos de retrieval e scoring com base em interações bem-sucedidas.
- 6. **Iniciar expansão N5** — Criar `scripts/analyze_n4_expansion.py` e `scripts/generate_n5.py`. Gerar 486 micro-regiões com assinaturas vetoriais distintas.
- 7. **Camada de Expressão Cognitiva** — Implementar meta-cognitive analysis e adaptive compression para modulação expressiva.

Longo Prazo (3+ meses)

- 8. **Fractal Storm Engine** — Substrato operacional completo com resonance dynamics, attractor fields e anti-colisão semântica.
- 9. **Semantic Cognitive Infrastructure Stack** — Memória viva, temporalidade cognitiva, simulação inferencial, ecossistema multi-agente.
- 10. **Avaliação 10/10** — Transição de "Teoria Brilhante" para "Produto Impecável" conforme definido em `roadmap.md`.

13. DECISÕES ARQUITETURAIS REGISTRADAS

#	Decisão	Justificativa	Data
1	Dual ID system (<code>uid</code> + <code>path</code> + <code>hierarchical_id</code>)	Imutabilidade para código + navegação estrutural + reconstrução de árvore	2026-05
2	Retrieval híbrido 0.5/0.3/0.2	Balanceamento entre precisão vetorial, contexto relacional e filtragem simbólica	2026-05
3	9 dimensões de assinatura semântica	Cobertura multidimensional sem sobreposição: abstracao, complexidade, causalidade, emocionalidade, materialidade, simbolismo, dinamismo, temporalidade, ambiguidade	2026-05
4	10 tipos de relação ponderados	Vocabulário controlado com pesos semânticos e reversões automáticas	2026-05
5	Pipeline explícito (11 etapas)	Rastreabilidade total: <code>normalize</code> → <code>tokenize</code> → <code>cleanup</code> → <code>lexical</code> → <code>embedding</code> → <code>scoring</code> → <code>graph</code> → <code>calibration</code> → <code>ambiguity</code> → <code>ranking</code> → <code>output</code>	2026-05
6	3 modos de operação (Full/Simple/Autonomous)	Trade-off entre profundidade e velocidade	2026-05
7	Config via YAML, não hardcoded	Separação de configuração e código, proteção de arquivos críticos	2026-05
8	<code>calibration_engine</code> absorvido no <code>validator</code>	Evita módulo órfão; calibração é verificação pré-output	2026-05
9	<code>cognition_engine</code> delegado a <code>cognitive_function_engine</code>	Coesão: funções cognitivas são o mecanismo cognitivo	2026-05
10	Memória via config YAML (não módulo Python)	Fase atual requer apenas configuração; implementação completa depende de feedback loop	2026-05

Nota de Consolidação: Este documento reflete o estado real do código base em 2026-05-13.

O código fonte (`core/`, `runtime/`, `tools/`, `data/`, `config/`) é a **única fonte de verdade**.

Os roadmaps originais continham 9 macro-componentes em `core/`; a implementação consolidou 4 deles em módulos existentes, resultando em 7 módulos operacionais.

As 4 camadas futuras (N5, Expressão Cognitiva, Fractal Storm Engine, Infrastructure Stack) permanecem como horizonte de evolução sem implementação atual.